Chapter 5.3: The Basics of Caches

Direct-Mapped Caches, Tags, and Write Policies

Contents

1 Introduction: What is a Cache?

Cache

Cache (pronounced "cash"): A safe place for hiding or storing things.
In computer architecture, a **cache** is:

- A small, fast storage level between the processor and main memory
- Any storage managed to take advantage of **locality of access**

The term "cache" was first used to describe the level of memory hierarchy between the processor and main memory. Today, it refers to any storage that exploits locality.

Historical Note: Caches first appeared in research computers in the early 1960s. Today, **every general-purpose computer** includes caches.

2 Direct-Mapped Cache

The fundamental question: *If a data item is in the cache, how do we find it?*

Direct-Mapped Cache

A cache structure in which each memory location is mapped to **exactly one location** in the cache.
Mapping Formula:

$$\text{Cache Block} = (\text{Block Address}) \mod (\text{Number of Blocks in Cache})$$

2.1 Simplification with Powers of 2

If the number of cache entries is a power of 2, the modulo operation becomes simple:

- Use the **low-order** $\log_2$**(cache size) bits** of the address
- Example: 8-block cache uses **3 lowest bits** ($8 = 2^3$)

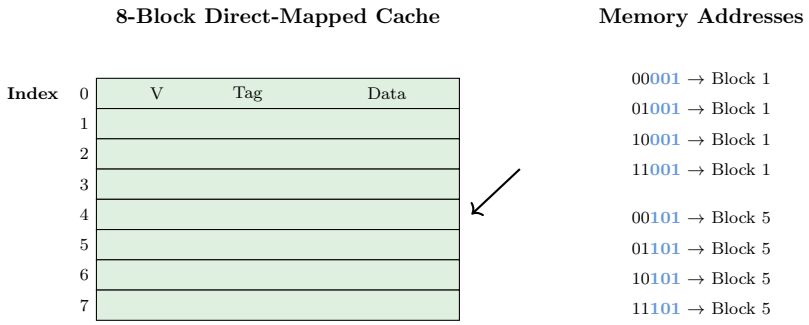

Figure 1: **Figure 5.8:** An 8-block direct-mapped cache. The 3 low-order bits determine the cache index.

3 Tags and Valid Bits

Since multiple memory addresses map to the same cache location, we need a way to identify *which* address is currently stored.

3.1 The Tag Field

Tag

A field in a cache entry that contains the **upper portion of the address** – the bits **not used** as the cache index.
The tag identifies whether the data in the cache corresponds to the requested word.

Why omit the index bits? They are **redundant**! By definition, the index field of any address in cache block i must equal i.

3.2 The Valid Bit

Valid Bit

A field that indicates whether a cache entry contains **valid data**.

- If valid bit = 0: Entry is invalid (ignore the tag)
- If valid bit = 1: Entry contains valid data

Why needed?

- When processor starts up, cache is empty (tags are meaningless)
- After running, some entries may still be empty
- Prevents false matches with garbage data

4 Address Breakdown

A memory address is divided into fields for cache access:

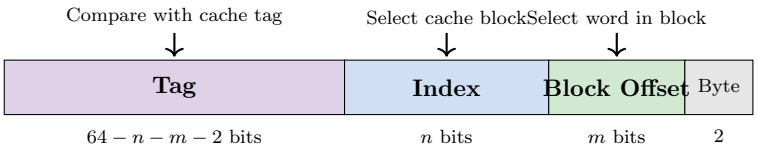

Figure 2: Address breakdown for a 64-bit address with 2^n blocks and 2^m words per block.

4.1 Cache Hit Conditions

Two Conditions for a Cache HIT

1. The **tag** in the cache entry must **match** the tag portion of the address
2. The entry's **valid bit** must be **ON** (= 1)

If either condition fails → **MISS**

5 Calculating Cache Size

5.1 Total Bits in a Cache

For a cache with:

- 64-bit addresses
- 2^n blocks
- 2^m words per block (block size = 2^{m+2} bytes)

Tag size: $64 - n - m - 2$ bits
Total bits:

$$2^n \times (2^m \times 32 + (64 - n - m - 2) + 1)$$

Example: Bits in a Cache

Problem: How many total bits for a direct-mapped cache with 16 KiB of data and 4-word blocks (64-bit address)?
Solution:

- 16 KiB = 4096 words = 2^{12} words
- Block size = 4 words = 2^2 words $\Rightarrow$ 1024 blocks = 2^{10}
- Each block: $4 \times 32 = 128$ bits of data
- Tag size: $64 - 10 - 2 - 2 = 50$ bits
- Total per block: $128 + 50 + 1 = 179$ bits
- Total: $1024 \times 179 = 183,296$ bits ≈ 22.4 KiB

Note: The physical size is 1.4× the "16 KiB" data capacity!

Convention: Cache size refers to **data only** (excludes tags and valid bits).

6 Block Size and Miss Rate

6.1 Benefits of Larger Blocks

Larger blocks exploit **spatial locality**:

- Fetching adjacent words along with requested word
- Usually **decreases miss rate**
- More efficient: less tag storage relative to data

6.2 Drawbacks of Larger Blocks

Problems with Very Large Blocks

1. **Increased Miss Penalty:** More time to transfer larger block
2. **Fewer Blocks:** If block size is too large relative to cache size, fewer blocks fit, increasing competition
3. **Diminishing Returns:** Spatial locality decreases with very large blocks

Design Guideline: The higher the memory bandwidth, the larger the cache block can be.

7 The Big Picture

The Big Picture: Caching as Prediction

Caching is perhaps the most important example of the big idea of **prediction**.

- It relies on **locality** to predict which data will be needed
- Provides mechanisms to handle **mispredictions** (fetch from lower levels)
- Modern cache **hit rates** are often **above 95%**!

8 Handling Cache Misses

Cache Miss

A request for data from the cache that **cannot be filled** because the data is not present in the cache.
Result: A **pipeline stall** – the processor freezes while waiting for memory.

8.1 Steps for Instruction Cache Miss

1. **Send address:** Send the original PC value to memory
2. **Wait:** Instruct main memory to perform a read; wait for completion
3. **Write to cache:**
   - Put data from memory in the data portion
   - Write upper address bits into the tag field
   - Set valid bit to ON
4. **Restart:** Re-execute the instruction (now it will hit in cache)

9 Handling Writes

Writes are more complex than reads because we must keep cache and memory **consistent**.

9.1 Write-Through

Write-Through

A scheme where writes **always update both** the cache and main memory.
Advantage: Simple; memory always has correct value
Disadvantage: Every write incurs full memory latency (∼100 cycles)

Performance Problem:

- If 10% of instructions are stores, CPI without misses = 1.0
- With 100-cycle write penalty: $\text{CPI} = 1.0 + 100 \times 0.10 = 11$
- **10× slower!**

9.2 Write Buffer

Write Buffer

A **queue** that holds data while waiting to be written to memory.

- Processor writes to cache AND write buffer, then continues
- Write buffer drains to memory in background
- Processor stalls only if write buffer is **full**

9.3 Write-Back

Write-Back

A scheme where writes update **only the cache**. The modified block is written to memory only when it is **replaced**.
Advantage: Fewer memory writes; better performance
Disadvantage: More complex; must track "dirty" blocks

9.4 Write Miss Policies

- **Write Allocate:** Fetch the block from memory, then overwrite the relevant word
- **No Write Allocate:** Update memory directly, don't bring block into cache

10 Reducing Miss Penalty

Early Restart

Resume execution as soon as the **requested word** arrives, without waiting for the entire block.

Critical Word First (Requested Word First)

Transfer the **requested word first** from memory, then send the rest of the block. The processor can restart immediately.

11 Example: Intrinsity FastMATH Processor

Intrinsity FastMATH Cache Organization

- **Split caches:** Separate instruction and data caches
- **Each cache:** 16 KiB, 256 blocks, 16 words/block
- **Address:** 32 bits
- **Tag:** 18 bits, **Index:** 8 bits, **Block offset:** 6 bits
- **Write policy:** Supports both write-through and write-back
- **Write buffer:** 1 entry
- **Pipeline:** 12 stages

Split Cache

A memory hierarchy level composed of **two independent caches**:

- One for **instructions**
- One for **data**

Operating in **parallel** to double cache bandwidth.

Trade-off: Split caches have slightly higher miss rate than a combined cache of the same total size, but the **doubled bandwidth** far outweighs this disadvantage.

12 Check Yourself

Which cache designer guidelines are generally valid?

1. The shorter the memory latency, the smaller the cache block
   Not generally true
2. The shorter the memory latency, the larger the cache block
   Not generally true
3. The higher the memory bandwidth, the smaller the cache block
   FALSE – opposite is true
4. **The higher the memory bandwidth, the larger the cache block**
   TRUE – high bandwidth can efficiently transfer larger blocks

13 Flashcards

1. **Q: What is a cache in computer architecture?**
   A: A small, fast storage level that acts as a safe place to store data needed by the processor, positioned between the processor and main memory.

2. **Q: What is a direct-mapped cache?**
   A: A cache structure in which each memory location is mapped to exactly one location in the cache.

3. **Q: What formula is used to map a memory address to a direct-mapped cache location?**
   A: (Block address) modulo (Number of blocks in the cache).

4. **Q: In an 8-block direct-mapped cache, how many address bits are used for the index?**
   A: 3 bits (since $\log_2(8) = 3$).

5. **Q: What is the purpose of the tag field in a cache entry?**
   A: It contains the upper portion of the address to identify whether the block corresponds to a requested word.

6. **Q: Why are index bits omitted from the cache tag?**
   A: They are redundant – the index field by definition must match the cache block number being accessed.

7. **Q: What is the purpose of the valid bit in a cache entry?**
   A: It indicates whether the associated block in the cache contains valid data.

8. **Q: What two conditions must be met for a cache HIT?**
   A: The tag must match AND the valid bit must be on.

9. **Q: Caching is the most important example of what computer architecture concept?**
   A: Prediction.

10. **Q: What is a cache miss?**
    A: A request for data from the cache that cannot be filled because the data is not present.

11. **Q: What happens to the processor during a cache miss?**
    A: It experiences a pipeline stall, freezing while waiting for memory.

12. **Q: What are the four steps for handling an instruction cache miss?**
    A: (1) Send PC to memory, (2) Wait for read to complete, (3) Write data/tag/valid to cache, (4) Restart instruction execution.

13. **Q: What is the write-through scheme?**
    A: A scheme where writes always update both the cache and main memory.

14. **Q: What is the primary drawback of write-through?**
    A: Every write incurs the long latency of main memory access.

15. **Q: What is a write buffer?**
    A: A queue that holds data while waiting to be written to memory, allowing the processor to continue execution.

16. **Q: What is the write-back scheme?**
    A: A scheme where writes update only the cache; the modified block is written to memory when replaced.

17. **Q: What is the write allocate policy?**
    A: On a write miss, fetch the block from memory and place it in the cache, then overwrite the relevant word.

18. **Q: What is the no write allocate policy?**
    A: On a write miss, update memory directly without bringing the block into the cache.

19. **Q: What is a split cache?**
    A: A memory hierarchy level with two independent caches operating in parallel: one for instructions, one for data.

20. **Q: What is the main advantage of split caches?**
    A: They double cache bandwidth by allowing simultaneous instruction and data accesses.

21. **Q: What principle of locality do larger cache blocks exploit?**
A: Spatial locality.

22. **Q: What is the drawback of making cache blocks too large?**
A: Increased miss penalty and higher miss rate due to fewer available blocks.

23. **Q: What design guideline relates memory bandwidth to cache block size?**
A: The higher the memory bandwidth, the larger the cache block can be.

24. **Q: What is early restart?**
A: A technique to reduce miss penalty by resuming execution as soon as the requested word arrives.

25. **Q: What is critical word first (requested word first)?**
A: A technique where memory transfers the requested word first, then sends the rest of the block.

26. **Q: For a 64-bit address with 2^n blocks and 2^m words per block, what is the tag size?**
A: $64 - n - m - 2$ bits.

27. **Q: Why must the number of entries in a direct-mapped cache be a power of 2?**
A: Because the index field is used as an address, and an n-bit field has 2^n possible values.

28. **Q: In the Intrinsity FastMATH processor, what are the cache specifications?**
A: 16 KiB each for instruction and data, 256 blocks, 16 words per block.

29. **Q: What write policies does the Intrinsity FastMATH support?**
A: Both write-through and write-back, selectable by the operating system.

30. **Q: When stating cache size (e.g., "16 KiB cache"), what is excluded?**
A: The storage for tags and valid bits – only data storage is counted.

31. **Q: For a cache with 64 blocks and 16-byte blocks, to what block does byte address 1200 map?**
A: Block 11 (calculated as $\lfloor 1200/16 \rfloor \mod 64 = 75 \mod 64 = 11$).

32. **Q: What is the typical hit rate of caches on modern computers?**
A: Often above 95%.

33. **Q: If cache and memory have different values for the same address, they are said to be...?**
A: Inconsistent.

34. **Q: Why can't a write-back cache overwrite a block before confirming a hit?**
A: Doing so could destroy the only valid copy of modified data.

35. **Q: What principle of locality is exploited when a recently referenced item replaces a less recently referenced one?**
A: Temporal locality.

36. **Q: In a direct-mapped cache, what determines which block gets replaced on a miss?**
A: There is no choice – the new item must go in the one location it maps to.

37. **Q: How does a write buffer improve performance in write-through caches?**
A: It allows the processor to continue execution while writes are pending to memory.

38. **Q: Under what condition must the processor stall when using a write buffer?**
A: When the write buffer is full.

39. **Q: How is the full memory address reconstructed from a cache entry?**
A: By concatenating the tag field with the index field (and block/byte offsets).

40. **Q: What is the trade-off of split caches vs. combined caches?**
A: Split caches have slightly higher miss rate but double the bandwidth.